Lab 3: PyTorch Fundamentals

Objective:

1. To learn and understand about basics of Pytorch for future implementation.

Theory:

PyTorch:

PyTorch is an open-source machine learning framework that is known for its flexibility, ease of use, and performance in modern AI applications. This is enabled in part by its compatibility with the popular Python high-level programming language favored by machine learning developers and AI researchers.

Tensors:

Tensors are the fundamental building block of PyTorch. Similar to multidimensional arrays or NumPy's ndarrays, tensors store and manipulate model inputs, outputs, and parameters. Crucially, PyTorch tensors are designed to run on NVIDIA GPUs, enabling massive parallel computation to accelerate training and inference.

Scalar:

Scalar is a single numerical value that conveys magnitude but no direction or dimension. It is a zero-dimensional entity, meaning it cannot be decomposed into smaller parts or represented along any axis. Scalars serve as the fundamental units of computation in mathematics, physics and computer science.

Vector:

A vector is a one-dimensional array of numerical values representing both magnitude and direction in space. Each element in a vector corresponds to a specific dimension, making vectors useful for encoding quantities that have directional components such as velocity, force or gradients.

Matrix:

A matrix is a two-dimensional grid or rectangular array of numbers arranged in rows and columns. It provides a structured representation of data where each row typically denotes an observation and each column denotes a feature.

Operations:

Impporting pYTorch

In [213…
```
import torch
torch.__version__
```

Out[213…
```
'2.12.0+cpu'
```

Scalar

In [214…
```
scalar = torch.tensor(7)
scalar
```

Out[214…
```
tensor(7)
```

In [215…
```
print(scalar.ndim)
print(scalar.item()) #only works with one-element tensors
print(scalar.shape)
```

```
0
7
torch.Size([])
```

Vectors

In [216…
```
vector = torch.tensor([7, 8, 9, 10])
vector
```

Out[216…
```
tensor([ 7,  8,  9, 10])
```

In [217…
```
print(vector.ndim)
print(vector.shape)
```

```
1
torch.Size([4])
```

Matrix

In [218…
```
matrix = torch.tensor([[1, 2, 3],
                       [4, 5, 6],
                       [7, 8, 9]])
matrix
```

Out[218…
```
tensor([[1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]])
```

In [219…
```
print(matrix.ndim)
print(matrix.shape)
```

```
2
torch.Size([3, 3])
```

Tensor

In [220…
```
TENSOR = torch.tensor([[[1, 2, 3],
                        [3, 6, 9],
```

```
                        [2, 4, 5]]])
TENSOR
```

Out[220…
```
tensor([[[1, 2, 3],
         [3, 6, 9],
         [2, 4, 5]]])
```

In [221…
```
print(TENSOR.ndim)
print(TENSOR.shape)
```

```
3
torch.Size([1, 3, 3])
```

The above tensor can represent, for example, the following datasets:

Day of the week	Chiya Sales	Coffee Sales
1	3	2
2	6	4
3	9	5

Transposed Form

Alternatively, we can write it in transposed form:

Day of the week	1	2	3
Chiya Sales	3	6	9
Coffee Sales	2	4	5

random tensors

In [222…
```
random_tensor = torch.rand(size=(2, 3))
random_tensor, random_tensor.dtype
```

Out[222…
```
(tensor([[0.0895, 0.3363, 0.3783],
         [0.2781, 0.0141, 0.8081]]),
 torch.float32)
```

In [223…
```
random_image_size_tensor = torch.rand(size=(224, 224, 3))
random_image_size_tensor.shape, random_image_size_tensor.ndim
```

Out[223…
```
(torch.Size([224, 224, 3]), 3)
```

tensors of only Zeros and Ones

In [224…
```
zeros = torch.zeros(size=(3, 4))
zeros, zeros.dtype
```

Out[224…
```
(tensor([[0., 0., 0., 0.],
         [0., 0., 0., 0.],
         [0., 0., 0., 0.]]),
 torch.float32)
```

In [225…
```
ones = torch.ones(size=(3, 4))
ones, ones.dtype
```

Out[225…
```
(tensor([[1., 1., 1., 1.],
         [1., 1., 1., 1.],
         [1., 1., 1., 1.]]),
 torch.float32)
```

creating a range and tensors like

In [226…
```
zero_to_ten = torch.arange(start=0, end=10, step=1)
zero_to_ten
```

Out[226…
```
tensor([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

In [227…
```
ten_zeros = torch.zeros_like(input=zero_to_ten)
ten_zeros
```

Out[227…
```
tensor([0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
```

Tensor datatypes

PyTorch offers many tensor data types, some optimized for CPUs and others for GPUs. Tensors associated with GPUs often use torch.cuda in their type or device specification. The default and most commonly used data type is torch.float32 (or torch.float), which represents 32-bit floating-point numbers.

Other floating-point types include:

torch.float16 (torch.half) — 16-bit floating point

torch.float64 (torch.double) — 64-bit floating point

PyTorch also supports integer types with different precisions, such as:

8-bit integers

16-bit integers

32-bit integers

64-bit integers

In [228…
```
float_32_tensor = torch.tensor([2.0, 4.0, 6.0],
                               dtype=None,
                               device=None,requires_grad=False)
```

```
float_32_tensor.shape, float_32_tensor.dtype, float_32_tensor.device
```

Out[228…
```
(torch.Size([3]), torch.float32, device(type='cpu'))
```

In [229…
```
float_16_tensor = torch.tensor([3.0, 6.0, 9.0],
                               dtype=torch.float16)

float_16_tensor.dtype
```

Out[229…
```
torch.float16
```

Getting information from tensors

The three of the most common attributes you'll want to find out about tensors are:

shape - what shape is the tensor? (some operations require specific shape rules)

dtype - what datatype are the elements within the tensor stored in?

device - what device is the tensor stored on? (usually GPU or CPU)

In [230…
```
some_tensor = torch.rand(3, 4)

print(some_tensor)
print(f"Shape of tensor: {some_tensor.shape}")
print(f"Datatype of tensor: {some_tensor.dtype}")
print(f"Device tensor is stored on: {some_tensor.device}")
```

```
tensor([[0.3645, 0.6584, 0.3005, 0.2252],
        [0.6674, 0.6382, 0.0969, 0.6463],
        [0.7791, 0.8631, 0.9639, 0.0031]])
Shape of tensor: torch.Size([3, 4])
Datatype of tensor: torch.float32
Device tensor is stored on: cpu
```

Tensor Operations:

In deep learning, different types of data such as images, text, audio, videos, and other information are converted into tensors (multi-dimensional arrays of numbers).

A model learns by repeatedly performing mathematical operations on these tensors to discover patterns and relationships in the data. During training, millions of calculations may be carried out.

Some of the most common tensor operations include:

Addition

Subtraction

Multiplication (element-wise)

Division

Matrix multiplication

In [231…
```
tensor = torch.tensor([1, 2, 3])
tensor + 10
```

Out[231…
```
tensor([11, 12, 13])
```

In [232…
```
tensor * 10
```

Out[232…
```
tensor([10, 20, 30])
```

In [233…
```
tensor
```

Out[233…
```
tensor([1, 2, 3])
```

In [234…
```
tensor = tensor - 10
tensor
```

Out[234…
```
tensor([-9, -8, -7])
```

In [235…
```
torch.multiply(tensor, 10)
```

Out[235…
```
tensor([-90, -80, -70])
```

In [236…
```
tensor
```

Out[236…
```
tensor([-9, -8, -7])
```

In [237…
```
tensor = tensor + 10
# Element-wise multiplication (each element multiplies its equivalent, index 0->0,
print(tensor, "*", tensor)
print("Equals:", tensor * tensor)
print("Equals:", torch.multiply(tensor, tensor))
```

```
tensor([1, 2, 3]) * tensor([1, 2, 3])
Equals: tensor([1, 4, 9])
Equals: tensor([1, 4, 9])
```

In [238…
```
# Let's create a tensor and perform element-wise multiplication and matrix multipli
tensor = torch.tensor([1, 2, 3])

# Element-wise matrix multiplication
print(tensor * tensor)

# Matrix multiplication
print(torch.matmul(tensor, tensor))
```

```
tensor([1, 4, 9])
tensor(14)
```

In [239…
```
# # Shapes need to be in the right way
tensor_A = torch.tensor([[1, 2],
```

```
                          [3, 4],
                          [5, 6]], dtype=torch.float32)

tensor_B = torch.tensor([[7, 10],
                         [8, 11],
                         [9, 12]], dtype=torch.float32)

# (this will error)
```

In [240…

```
print(f"Original shapes: tensor_A = {tensor_A.shape}, tensor_B = {tensor_B.shape}\n
print(f"New shapes: tensor_A = {tensor_A.shape} (same as above), tensor_B.T = {tens
print(f"Multiplying: {tensor_A.shape} * {tensor_B.T.shape} <- inner dimensions matc
print("Output:\n")
output = torch.matmul(tensor_A, tensor_B.T)
print(output)
print(f"\nOutput shape: {output.shape}")
```

```
Original shapes: tensor_A = torch.Size([3, 2]), tensor_B = torch.Size([3, 2])

New shapes: tensor_A = torch.Size([3, 2]) (same as above), tensor_B.T = torch.Size
([2, 3])

Multiplying: torch.Size([3, 2]) * torch.Size([2, 3]) <- inner dimensions match

Output:

tensor([[ 27.,  30.,  33.],
        [ 61.,  68.,  75.],
        [ 95., 106., 117.]])

Output shape: torch.Size([3, 3])
```

Finding the min, max, mean, sum, etc (aggregation)

In [241…

```
x = torch.arange(0, 100, 10)
print(x)
print(f"Minimum: {x.min()}")
print(f"Maximum: {x.max()}")
print(f"Mean: {x.type(torch.float32).mean()}")
print(f"Sum: {x.sum()}")
```

```
tensor([ 0, 10, 20, 30, 40, 50, 60, 70, 80, 90])
Minimum: 0
Maximum: 90
Mean: 45.0
Sum: 450
```

Positional min/max

In [242…

```
# Create a tensor
tensor = torch.arange(10, 100, 10)
print(f"Tensor: {tensor}")

# Returns index of max and min values
print(f"Index where max value occurs: {tensor.argmax()}")
print(f"Index where min value occurs: {tensor.argmin()}")
```

```
Tensor: tensor([10, 20, 30, 40, 50, 60, 70, 80, 90])
Index where max value occurs: 8
Index where min value occurs: 0
```

Changing tensor datatype

In [243…
```
# Create a tensor and check its datatype
tensor = torch.arange(10., 100., 10.)
print(tensor.dtype)

# Create a float16 tensor
tensor_float16 = tensor.type(torch.float16)
tensor_float16
```

```
torch.float32
```

Out[243…
```
tensor([10., 20., 30., 40., 50., 60., 70., 80., 90.], dtype=torch.float16)
```

Reshaping

In [244…
```
# Create a tensor
import torch
x = torch.arange(1., 9.)
print(x, x.shape)

# Add an extra dimension
x_reshaped = x.reshape(1, 8) # try other shapes, it will produce an error.
x_reshaped, x_reshaped.shape
```

```
tensor([1., 2., 3., 4., 5., 6., 7., 8.]) torch.Size([8])
```

Out[244…
```
(tensor([[1., 2., 3., 4., 5., 6., 7., 8.]]), torch.Size([1, 8]))
```

In [245…
```
x, x.shape
```

Out[245…
```
(tensor([1., 2., 3., 4., 5., 6., 7., 8.]), torch.Size([8]))
```

View

In [246…
```
z = x.view(1, 8)
z, z.shape
```

Out[246…
```
(tensor([[1., 2., 3., 4., 5., 6., 7., 8.]]), torch.Size([1, 8]))
```

In [247…
```
z = x.view(2,4)
z, z.shape
```

Out[247…
```
(tensor([[1., 2., 3., 4.],
         [5., 6., 7., 8.]]),
 torch.Size([2, 4]))
```

In [248…
```
x, x.shape
```

Out[248…
```
(tensor([1., 2., 3., 4., 5., 6., 7., 8.]), torch.Size([8]))
```

In [249…
```
import torch
x = torch.arange(6)
# print(x,x.shape)

a = x.view(2, 3)
b = x.reshape(2, 3)

a[0, 0] = 100
print(x)
```

```
tensor([100,   1,   2,   3,   4,   5])
```

That's the exact reason both functions exist: view() is a strict "reinterpret the existing memory" operation, while reshape() is a flexible "give me this shape, even if a copy is needed" operation.

Stack

In [250…
```
x_stacked = torch.stack([x, x, x, x], dim=0)
x_stacked
```

Out[250…
```
tensor([[100,   1,   2,   3,   4,   5],
        [100,   1,   2,   3,   4,   5],
        [100,   1,   2,   3,   4,   5],
        [100,   1,   2,   3,   4,   5]])
```

In [251…
```
x_stacked = torch.stack([x, x, x, x], dim=1)
x_stacked
```

Out[251…
```
tensor([[100, 100, 100, 100],
        [  1,   1,   1,   1],
        [  2,   2,   2,   2],
        [  3,   3,   3,   3],
        [  4,   4,   4,   4],
        [  5,   5,   5,   5]])
```

Squeeze and Unsqueeze funtions.

Because deep learning models (neural networks) are all about manipulating tensors in some way. And because of the rules of matrix multiplication, if you've got shape mismatches, you'll run into errors. These methods help you make sure the right elements of your tensors are mixing with the right elements of other tensors.

In [252…
```
print(f"Previous tensor: {x_reshaped}")
print(f"Previous shape: {x_reshaped.shape}")

# Remove extra dimension from x_reshaped
x_squeezed = x_reshaped.squeeze()
print(f"\nNew tensor: {x_squeezed}")
print(f"New shape: {x_squeezed.shape}")

## Add an extra dimension with unsqueeze
x_unsqueezed = x_squeezed.unsqueeze(dim=0)
```

```
print(f"\nNew tensor: {x_unsqueezed}")
print(f"New shape: {x_unsqueezed.shape}")
```

```
Previous tensor: tensor([[1., 2., 3., 4., 5., 6., 7., 8.]])
Previous shape: torch.Size([1, 8])

New tensor: tensor([1., 2., 3., 4., 5., 6., 7., 8.])
New shape: torch.Size([8])

New tensor: tensor([[1., 2., 3., 4., 5., 6., 7., 8.]])
New shape: torch.Size([1, 8])
```

permute

In [253…

```
import torch

x = torch.tensor([[1, 2, 3],
                  [4, 5, 6]])

print(x.shape)

y = x.permute(1, 0)

print(y)
print(y.shape)
```

```
torch.Size([2, 3])
tensor([[1, 4],
        [2, 5],
        [3, 6]])
torch.Size([3, 2])
```

Note: Because permuting returns a view (shares the same data as the original), the values in the permuted tensor will be the same as the original tensor and if you change the values in the view, it will change the values of the original

Indexing

In [254…

```
import torch
x = torch.arange(1, 10).reshape(1, 3, 3)
x, x.shape

print(f"First square bracket:\n{x[0]}")
print(f"Second square bracket: {x[0][0]}")
print(f"Third square bracket: {x[0][0][0]}")
```

```
First square bracket:
tensor([[1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]])
Second square bracket: tensor([1, 2, 3])
Third square bracket: 1
```

In [255…

```
print(x[:, 0]) # Get all values of 0th dimension and the 0 index of 1st dimension
print(x[:, :, 1])# Get all values of 0th & 1st dimensions but only index 1 of 2nd d
```

```
print(x[:, 1, 1]) # Get all values of the 0 dimension but only the 1 index value of
print(x[0, 0, :]) # same as x[0][0]
```

```
tensor([[1, 2, 3]])
tensor([[2, 5, 8]])
tensor([5])
tensor([1, 2, 3])
```

Pytorch Tensor to NumPy arrays and vice versa

In [256…
```
# NumPy array to tensor
import torch
import numpy as np
array = np.arange(1.0, 8.0)
tensor = torch.from_numpy(array)
array, tensor
```

Out[256…
```
(array([1., 2., 3., 4., 5., 6., 7.]),
 tensor([1., 2., 3., 4., 5., 6., 7.], dtype=torch.float64))
```

In [257…
```
# Tensor to NumPy array
tensor = torch.ones(7) # create a tensor of ones with dtype=float32
numpy_tensor = tensor.numpy() # will be dtype=float32 unless changed
tensor, numpy_tensor
```

Out[257…
```
(tensor([1., 1., 1., 1., 1., 1., 1.]),
 array([1., 1., 1., 1., 1., 1., 1.], dtype=float32))
```

Tensors on GPU

In [258…
```
!nvidia-smi
```

```
Wed Jun 10 23:14:25 2026
+-----------------------------------------------------------------------------------------
------+
| NVIDIA-SMI 581.32                 Driver Version: 581.32         CUDA Version: 13.
0     |
+-----------------------------------------+------------------------+----------------
------+
| GPU  Name                  Driver-Model | Bus-Id          Disp.A | Volatile Uncor
r. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compu
te M. |
|                                         |                        |               M
IG M. |
|=========================================+========================+================
======|
|   0  NVIDIA GeForce MX350         WDDM  |   00000000:01:00.0 Off |
N/A |
| N/A   61C    P0            N/A  / 5001W |       0MiB /   2048MiB |      0%      De
fault |
|                                         |                        |
N/A |
+-----------------------------------------+------------------------+----------------
------+

+-----------------------------------------------------------------------------------------
------+
| Processes:
|
|  GPU   GI   CI              PID   Type   Process name                        GPU M
emory |
|        ID   ID                                                               Usage
|
|=========================================================================================
======|
|  No running processes found
|
+-----------------------------------------------------------------------------------------
------+
```

In [259…
```
import torch
torch.cuda.is_available()
```

Out[259…
```
False
```

In [260…
```
device = "cuda" if torch.cuda.is_available() else "cpu"
device
```

Out[260…
```
'cpu'
```

In [261…
```
torch.cuda.device_count()
```

Out[261…
```
0
```

In [262…
```
# Create tensor (default on CPU)
tensor = torch.tensor([1, 2, 3])
```

```
# Tensor not on GPU
print(tensor, tensor.device)

# Move tensor to GPU (if available)
tensor_on_gpu = tensor.to(device)
tensor_on_gpu
```

```
tensor([1, 2, 3]) cpu
```

Out[262… tensor([1, 2, 3])

In [263…
```
# Instead, copy the tensor back to cpu
tensor_back_on_cpu = tensor_on_gpu.cpu().numpy()
tensor_back_on_cpu
```

Out[263… array([1, 2, 3])

Discussion & Conclusion

In this lab, we learned about fundamentals of PyTorch,from its installation to advanced tensor manipulation. we learned about mathematical structures involved like, Scalar, vector, matrices, and our major focus was on tensors. use of various tensor attributes highlighted how pyTorch dynamically tracks dimensions. We also used matrix multiplication, whichi is considered computational backbone of neural networking, this section emphasized the strict dimensional rules of matrix, the inner dimensions of interacting tensors must align perfectly, and the resulting tensor takes the shape of the outer dimensions. We also practiced using the transpose operation to dynamically flip dimensions and resolve shape mismatch errors. we further learned how to extract summary statistics from tensors using aggregation function. We then looked at methods for changing tensor dimensions without altering the underlying data. We contrasted operations like reshape,view, and permute. The final sections of the lab focused on integration and hardware execution. We demonstrated the seamless bridge between NumPy arrays and PyTorch tensors via torch.from_numpy and numpy, proving that PyTorch integrates cleanly into the broader Python data science ecosystem. At the end, we ckecked for gpu availability to move tensor from cpu.

In the conclusion, this lab exercise succesfully achieved its objective by establishing a foundational understanding of PyTorch tensor mecanics. tensors serve as the mathematical language required to represent, alter, and pass high-dimensional data through machine learning model pipelines.